Оглавление

1.1 - Базовая аннотация типов данных 3
2.1 - Области видимости переменных 6
2.2 - lambda-функции 8
2.3 - Замыкание функций 9
2.4 - Декораторы 10
2.5 - Декораторы с параметрами 13
2.6 - Сортировки с ключом key 15
3.4 - Виртуальное окружение....17
9.1 Множества: Особенности и основные характеристики 18
Множества....18
Зачем множества....18
Что значит «хешируемый» (hashable) объект....18
Что такое множество в Python....19
Создание множества....19
Вывод множества....19
Встроенная функция set()....19
Дубликаты при создании множества....20
Типы данных в множествах....20
Ограничения на кортежи в множествах....20
Частые ошибки и нюансы....20
Работа с f-строками....21
Заключение....21
9.2 Множества: Основные операции и функции 22
Работа с множествами....22
Основы работы с множествами....22
Функция len()....22
Оператор принадлежности in....22
Встроенные функции sum(), min(), max()....23
Перебор множества....23
Форматированный вывод множеств....23
Сравнение множеств....24
Функция sorted()....24
Сортировка с помощью key....24
Обратная сортировка (reverse=True)....25
Заключение....25
9.3 Методы добавления и удаления элементов в множества 26
Методы множеств....26
Добавление элементов....26
Удаление элементов....26
Метод очистки множества....27
Заключение....27
9.4 Методы объединения, пересечения и разности множеств 28
Операции и методы множеств....28
Операции над множествами....28
Объединение множеств: метод union()....28
Пересечение множеств: метод intersection()....28
Разность множеств: метод difference()....29
Симметрическая разность: метод symmetric_difference()....29

Методы для изменения множества....30
Метод update()....30
Метод intersection_update()....30
Метод difference_update()....31
Метод symmetric_difference_update()....31
Множества в Python, с их определениями и операторами:....32
В виде таблицы:....32
Заключение....33
9.5 Методы множеств: Подмножества, Надмножества и Сравнение 34
Методы множеств и их сравнение....34
Что такое подмножества и надмножества....34
Метод issubset(): проверка подмножества....34
Метод issuperset(): проверка надмножества....34
Метод isdisjoint(): проверка на отсутствие общих элементов....35
Сравнение множества с другими итерируемыми объектами....35
Соответствия методов и операторов....35
? В виде таблицы соответствия методов и операторов....36
Пояснения:....36
Заключение....36

1.1 - Базовая аннотация типов данных

Библиотека **mypy**(для проверки корректности типизации в коде):

```
# Установка
pip install mypy

# Проверка нашего файла в корне проекта:
mypy .\имя файла
```

Аннотация переменной(редко юзается):

```
x: int
x = 'gh'

# или

x: int = 'gh'
```

Пример типизации параметра и возвращаемого объекта в функции:

```
def func(n: int) -> int:  # n: int = 5 - для именнованного параметра
    return sum(range(1, n + 1))
```

У функций при аннотации появляется магический атрибут **__annotations__**, который по сути является словарем, в котором указаны все аннотированные параметры и **return**:

```
def func(n: int = 5) -> int:
    return sum(range(1, n + 1))

print(func.__annotations__)
```

Если функция не возвращающая, то для **return** указываем **None** :

```
def func() -> None:
    print('Буги-Вуги')
```

Если наша функция подразумевает прием или возврат разных типов, для этого используется модуль **typing**, оттуда импортируется тип **Union**:

```
from typing import Union


def func(x: Union[int, float], y: Union[int, float]) -> Union[int, float]:
    return x + y
```

Начиная с версии Python 3.10 необходимость этого импорта отпала, теперь объединять типы можно так:

```
def func(x: int | float, y: int | float) -> int | float:
    return x + y
```

Типы аннотаций можно сохранять в переменные, обычно это делается когда аннотация немного сложнее чем обычно:

```
from typing import Union

z = Union[int, float]
al = int | float


def func(x: z, y: al) -> int:
    return x + y
```

Если параметр или возвращаемое значение подразумевает какой-то тип или **None**, то используется тип **Optional** из модуля **typing**:

```
from typing import Optional


def func(x: list, y: list) -> Optional[list]: # list | None
    match x, y:
        case list(), list():
            return x + y
    return None # здесь без явного возврата None mypy ругается
```

Соответственно можно использовать **Union[list, None]**, но это менее удобный вариант, поэтому и собственно был придуман
Optional. Но вполне себе удобный вариант => **list | None.**

Если мы не можем определиться какой тип данных необходимо принять функции или вернуть, то в таком случае используется
тип **Any** из модуля **typing**:

```
from typing import Any


def func(*args: Any) -> list:
    return list(args)
```

Если вы хотите создать переменную-константу, значение которой менять нельзя можно использовать тип **Final** модуля **typing**:

```
from typing import Final

password: Final = 'qwerty'
password = 'пароль'
```

При аннотировании списков мы можем указать какие-типы данных могут быть внутри:

```
lst: list[int | str] = [1, 2, 3, '4']
```

До версии Python 3.9 коллекции также импортировались из модуля **typing** как типы: **List, Tuple, Dict, Set**. Поэтому этот код можно переписать так:

```
from typing import List
```

```
lst: List[int | str] = [1, 2, 3, '4']
```

Для кортежей мы должны указывать тип для каждого его элемента, почему так? Дело в том, что те же списки обычно используются для объединения объектов одного типа, а кортежи для разных - отсюда и различия:

```
tpl_1: tuple[int, str] = (1, 'Альфа')
```

Но если все же у нас будут объекты одного типа, то это можно записать так:

```
tpl_2: tuple[int, ...] = (1, 2, 3)
```

Если мы хотим аннотировать словари, то в квадратных скобках нужно указать какие типы данных мы ожидаем от ключей и значений, через запятую:

```
dct: dict[int, str] = {1: 'Аня', 2: 'Никита'}
```

Для множеств также как и для списков указывают один тип для всех элементов:

```
st: set[int] = {1, 3, 5, 9}
```

Пример вложенности аннотаций:

```
from typing import Any


def func(data: list[str | int] | dict[int, Any]) -> set[int |
str]:
    return set(data)
```

Если необходимо указать конкретное значение типа, то нужно использовать **Literal**:

```
from typing import Literal


def func(tpl: tuple[int, Literal[5]]) -> None:
    pass


print(func((3, 5)))
```

Если мы хотим использовать вызываемый тип данных, например функция или метод, то используется тип **Callable** модуля **collections.abc**, в нем в **[]** мы можем первым аргументом указать параметр(тоже в **[]**), затем возвращаемый тип(без **[]**):

```
from collections.abc import Callable

def func(f: Callable[[str], int], lst: list[str]) -> list[int]:
    if lst is None:
        return []
    return [f(i) for i in lst]
```

```
print(func(len, ['Abc', 'Bcdef']))
```

2.1 - Области видимости переменных

В примере ниже переменные **args**, **summ**, **i** являются локальными, их областью видимости является тело функции:

```
def func(*args):
    summ = 0
    for i in args:
        summ += i
    return summ
```

Здесь мы добавили переменную **x** и немного изменили функцию, функция из локальной области видимости использует значение глобальной переменной, т.к. в своей области имен одноименная переменная не объявлялась. Т.е. в любой вложенной области имен, если есть запрос на переменную и она в данной области отсутствует, то происходит поиск данной переменной областью выше:

```
x = 100

def func(*args):
    summ = 0
    for i in args:
        summ += i
    return summ + x

print(func(5, 5))
```

Если одноименная переменная существует во всех областях видимости, то берется значение из той же области в которой происходит запрос на данную переменную:

```
x = 100

def func(*args):
    summ = 0
    x = 20
    for i in args:
        summ += i
    return summ + x

print(func(5, 5))
```

Если вы все же хотите изменять значения переменных глобальной области из вложенной области, то нужно использовать ключевое слово **global**:

```
x = 100
y = 500


def func(*args):
    global x, y # здесь можно перечислять переменные
    x = 20
    summ = 0
    for i in args:
        summ += i
    return summ + x


print(func(5, 5), x)
```

Пример в котором показано уже 3 области видимости, в каждой из них есть переменная **x** и все они независимы:

```
x = 1


def func_1():
    x = 2

    def func_2():
        x = 3
        print('x_func_2 =', x)
    func_2()
    print('x_func_1 =', x)


func_1()
print('global x =', x)
```

Если наша задача влиять из локальной области имен на переменные локальной области уровнем выше, то необходимо использовать ключевое слово **nonlocal**:

```
x = 1


def func_1():
    x = 2

    def func_2():
        nonlocal x
        x = 3
        print('x_func_2 =', x)
    func_2()
    print('x_func_1 =', x)


func_1()
print('global x =', x)
```

Из локальной области можно влиять на глобальные переменные и без **global**. Если внести изменения на изменяемый объект, например список. Т.к. в данном случае не происходит замена объекта как такового, происходит только его изменение:

```
lst = [1, 2, 3]


def func():
    lst.append(4)


func()
print(lst)
```

2.2 - lambda-функции

Простой пример **lambda** функции:

```
x = lambda a, b: a + b
print(x(3, 7))
```

где **a**, **b** - параметры функции
a + b - return

lambda функция может быть без аргументов, но как правило так она не используется. Самый простой вариант:

```
print((lambda: 5)())
```

Пример функции, которая принимает необязательным аргументом **lambda** функцию, которая выполняет роль фильтра:

```
def func(lst, filtr=None):
    if filtr is None:
        return lst
    new_lst = [i for i in lst if filtr(i)]
    return new_lst


print(func([1, 2, 3, 4, 5, 12, 66], filtr=lambda x: 9 < x < 100))
# x % 2 == 0
```

В **lambda** функции можно использовать моржовый оператор:

```
z = lambda x: x + (y := 5)
print(z(8))
```

lambda функции также поддерживают все виды аргументов, что и обычные функции:

```
f1 = lambda x, y, z: x + y + z
f2 = lambda x, y, z=3: x + y + z
f3 = lambda *args: sum(args)
```

```
f4 = lambda **kwargs: sum(kwargs.values())
```

2.3 - Замыкание функций

В примере ниже представлено как внешняя функция объявляет и возвращает внутреннюю и когда казалось бы внешняя функция выполнила свою работу и больше никак не связана с внутренней, всё равно возвращенная функция пользуется пространством имен внешней:

```
def func_1(name):
    def func_2():
        print(f'Привет, {name}!')
    return func_2


x = func_1('Артем')
x()
```

Это поведение называется замыканием функций. Пространство имен внешней функции существует пока к нему сохраняется хоть одна ссылка. Цепочка вызовов: переменная **x** -> вложенная функция -> пространство имен внешней функции. Пока вложенная функция пользуется пространством имен внешней функции(переменная **name**) до тех пор существует связь между внешней и вложенной функцией, даже если внешняя функция отработала и вернула внутреннюю.

Пример в котором пространство имен внешней функции является своего рода буфером для внутренней функции. Ниже представлена функция счетчика:

```
def count(number):
    def step():
        nonlocal number
        number += 1
        return number
    return step


x = count(5)
print(x(), x(), x())
```

Здесь переменная **number** "паркуется" во внешнем пространстве имен, а внутренняя функция имеет доступ к ней с возможностью изменения при помощи **nonlocal**. Казалось бы мы пользуемся результатом внешней функции, т.е. функцией **step**, но как и было сказано некое пространство имен еще существует уровнем выше и оттуда и вытаскиваются необходимые данные.

Пример создания кэша максимум до **10** элементов во внешней области в виде списка:

```
def cash_lst():
    lst = []
```

```
    def add_cash(obj=None):
        if obj is None:
            return lst
        (lst.pop(0), lst.append(obj)) if len(lst) == 10 else
lst.append(obj)
        return lst

    return add_cash


x = cash_lst()
print(x(1))
print(x(2))
print(x(3))
print(x(4))
print(x(5))
print(x(6))
print(x(7))
print(x(8))
print(x(9))
print(x(10))
print(x(11))
print(x(12))
```

Кэш как и всегда расположен в пространстве имен внешней функции. Вызов внешней функции подразумевает создание внутренней, у которой есть доступ к пустому списку. Первые 10-ть вызовов с аргументом подразумевают пополнение данного списка и его **return**. Далее происходит пополнение с одновременным удалением первого элемента.

2.4 - Декораторы

Ниже пример создания декоратора, который применяется к функции и добавляет ей новый функционал в виде дополнительного вывода перед запуском функции и после:

```
def decorator(func):
    def wrapper():
        print('Начало ф-ии')
        func()
        print('Конец ф-ии')
    return wrapper


def funcx():
    print('Ф-ия func в работе...')
```

Чтобы применить декоратор к функции можно использовать несколько вариантов, самый распространенный:

```
def decorator(func):
```

```
    def wrapper():
        print('Начало ф-ии')
        func()
        print('Конец ф-ии')
    return wrapper


@decorator
def funcx():
    print('Ф-ия func в работе...')


funcx()
```

В данном варианте любой запуск функции **funcx** подразумевает запуск декорированного варианта.

Тоже самое, но в ручном варианте:

```
def decorator(func):
    def wrapper():
        print('Начало ф-ии')
        func()
        print('Конец ф-ии')
    return wrapper


def funcx():
    print('Ф-ия func в работе...')


funcx = decorator(funcx)
funcx()
```

Преимущество данного способа в том, если наша задача будет использовать оба варианта функции - с декоратором и без, здесь это возможно. Просто переменную **funcx** меняем на другую:

```
def decorator(func):
    def wrapper():
        print('Начало ф-ии')
        func()
        print('Конец ф-ии')
    return wrapper


def funcx():
    print('Ф-ия func в работе...')


funcx_ = decorator(funcx)
funcx_()
print('---------------------')
funcx()
```

Если декорируемая функция предполагает наличие атрибутов, то это записывается так:

```
def decorator(func):
    def wrapper(*args):
        print('Начало ф-ии')
        func(*args) # тут распаковка
        print('Конец ф-ии')
    return wrapper


@decorator
def funcx(name):
    print(f'Ф-ия func с атрибутом {name} в работе...')


funcx('abc')
```

Т.е. во **wrapper** мы прокидываем аргументы, которые собираются в кортеж, а при вызове функции он распаковывается. С **args** декоратор становится универсальным. Соответственно для именованных параметров мы будем использовать **kwargs**.

Пример применения декоратора для измерения времени работы функции:

```
import time


def time_decorator(func):
    def wrapper(*args, **kwargs):
        x_1 = time.time()
        f = func(*args, **kwargs)
        print(f'Время работы ф-ии: {time.time() - x_1}')
        return f
    return wrapper


# @time_decorator
def funcx(lst):
    s = 0
    for i in lst:
        s += i**3
    return s


funcx = time_decorator(funcx)    # эта конструкция аналогична
@time_decorator
print(funcx(range(1, 19999999)))
```

Декорирование функции возможно сразу несколькими функциями-декораторами:

```
def decor_1(func):
    def wrapper(*args):
        return f'-----------\n{func(*args)}\n-----------'
    return wrapper
```

```
def decor_2(func):
    def wrapper(*args):
        return f'**********\n{func(*args)}\n**********'
    return wrapper


@decor_2
@decor_1
def func_x(s):
    return s


print(func_x('Бармалей'))
```

Сначала функция декорируется декоратором decor_1, затем декорированная функция декорируется декоратором decor_2. Обратите внимание на порядок декорации в примере. В ручном варианте это выглядит так:

```
x = decor_2(decor_1(func_x))
print(x("Бармалей"))
```

2.5 - Декораторы с параметрами

Есть небольшая проблема, связанная с декорированием функций, оригинальная функция при декорировании теряет свое имя и документацию, если она есть:

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print(f'Начало работы ф-ии {func}')
        return func(*args, **kwargs)
    return wrapper


# @decorator
def func(it):
    """
    Эта ф-ия аналог sum
    :param it: iterable
    :return: int
    """
    return sum(it)


print(func, func.__doc__, func.__name__, sep='\n')
```

В данном примере функция еще не задекорирована. При стандартном выводе мы видим упоминание имени. Также имя функции можно вернуть с помощью специальной переменной **__name__**, документацию соответственно при помощи переменной **__doc__**. Теперь раскомментируйте декоратор и убедитесь, что изначальные данные потеряны.

Есть два способа исправить ситуацию. В первом случае, так как декоратор возвращает обертку, а она в свое время тоже функция, мы переменные имени и документации обертки присваиваем к переменным функции:

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print(f'Начало работы ф-ии {func}')
        return func(*args, **kwargs)
    wrapper.__name__ = func.__name__
    wrapper.__doc__ = func.__doc__
    return wrapper


@decorator
def func(it):
    """
    Эта ф-ия аналог sum
    :param it: iterable
    :return: int
    """
    return sum(it)


print(func, func.__doc__, func.__name__, sep='\n')
```

Второй(чаще используемый) воспользоваться специальным декоратором **wraps** из библиотеки **functools**:

```
from functools import wraps

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f'Начало работы ф-ии {func}')
        return func(*args, **kwargs)
    return wrapper


@decorator
def func(it):
    """
    Эта ф-ия аналог sum
    :param it: iterable
    :return: int
    """
    return sum(it)


print(func, func.__doc__, func.__name__, sep='\n')
```

Технически этот декоратор делает тоже самое, что и в первом случае.

Декоратор с параметром очень похож на обычный. Технически это тот же декоратор, только вложенный в еще одну функцию, суть которой принять параметр и "протащить" его в обертку:

```
def decor_param(typ):
    def decor(func):
        def wrapper(*args):
            if len([i for i in args if type(i) == typ]) !=
len(args):
                return f'Аргументами должны быть объекты типа -
{typ}'
            return func(*args)
        return wrapper
    return decor


@decor_param(str)
def func(a, b):
    return a + b


print(func('8', '5'))

# x = decor_param(int)(func)
# print(x(8, 5))
```

Здесь декоратор служит в качестве валидатора и является неким переключателем между типами, которые могут быть параметрами функции.

2.6 - Сортировки с ключом key

Прежде чем мы с вами поговорим о параметре **key**, напомню что функция **sorted** и метод **list.sort** отличаются тем, что в первой возвращается новый объект, а во втором видоизменяется список к которому был применен метод. А объединяет их этот самый параметр. Т.е. в зависимости от потребности используется какая-то из этих 2-ух сортировок, но принцип сортировки в них идентичный, в том числе и с параметром **key**.

Пример сортировки по модулю:

```
lst = [-5, 5, 3, -7, -2, 23]
print(sorted(lst, key=abs))
```

Принцип сортировки в том, что к каждому объекту нашей коллекции применяется функция из аргумента **key**, в данном случае функция **abs**, и уже получившиеся результаты сравниваются между собой. Но сортируются изначальные объекты, а не те которые мы получили после применения функции из параметра **key**.

Пример сортировки сначала по четным числам - потом по нечетным:

```
lst = [-5, 5, 3, -7, 24, 23, 6]
print(sorted(lst, key=lambda x: (x % 2, x)))
```

Это пример двойной сортировки, сначала применяется **x % 2**, в результате которой мы получаем **0** и **1**. Значения с **0** - четные числа и поэтому они попадут в начало коллекции(мы же сортируем по возрастающей). Второй аргумент в кортеже необходим, когда внутри группы четных или нечетных нужна дополнительная сортировка. В данном случае я хочу чтобы четные отсортировались между собой по возрастающей поэтому я и прописал просто **x**. Тоже самое происходит и внутри группы нечетных.

Пример сортировки по методу:

```
lst = ['B', 'a', 'd', 'C', 'r', 'o']
print(sorted(lst, key=str.lower))  # lambda x: x.lower()
```

С **lambda** это все таки понятнее выглядит.

Сортировка по длине слов и алфавиту:

```
lst = ['ремонт', 'арбалет', 'рыбий хрящь', 'эвакуатор', 'брикет',
'стирка']
lst.sort(key=lambda x: (len(x), x))
print(lst)
```

Пример сортировки вложенных коллекций по индексу элемента:

```
lst = [['Nintendo64', 2500],
       ['ps1', 4300],
       ['Dreamcast', 3700],
       ['Dendy', 1300]
       ]

print(sorted(lst, key=lambda x: x[1]))
```

Начну с того, что если мы сравниваем коллекции, а не простые объекты, то сравнение всегда происходит по первому элементу, попробуйте запустить код без ключа key. В нашем случае мы применяем функцию, которая возвращает второй элемент коллекции, а значит и сравнение происходит по нему. Если прописать **-x[1]**, то сравнение будет по убывающей.

3.4 - Виртуальное окружение

Команды:

```
# Создать виртуальное окружение в папке проекта

python -m venv venv


# Посмотреть список установленных интепретаторов

py --list           - для Windows
whereis python3     - для Linux и MacOs


# Создать виртуальное окружение с интерпретатором определенной
версии в папке проекта

py -3.7 -m venv venv - для Windows, где 3.7 версия python
python3.7 -m venv venv - для Linux и MacOs


# Активация виртуального окружения из папки проекта

venv\Scripts\activate.bat - для Windows
source venv/bin/activate - Для Mac и Linux


# Выйти из виртуального окружение

deactivate
```

9.1 Множества: Особенности и основные характеристики

Множества

Зачем множества

Представьте, что вы ведёте список посетителей на мероприятии. Люди могут подходить повторно, писать имя разными руками, а вам нужна простая честная проверка: «этот уже был?» Вот здесь множество работает как строгий контролёр на входе: оно хранит только уникальные значения и умеет очень быстро отвечать на вопрос о принадлежности.

Вторая типичная сцена выглядит ещё «математичнее». У вас есть две группы элементов, и вы хотите получить объединение, пересечение или разность, то есть операции в духе школьных множеств. И третья важная мысль: если порядок вообще не важен, а важна скорость поиска по элементу, то множество почти всегда оказывается естественным выбором.

Что значит «хешируемый» (hashable) объект

Немного отступим от урока и поговорим о **hashable.** Представьте, что множество устроено как очень быстрый каталог в библиотеке. Чтобы находить элемент почти мгновенно, ему нужен «паспорт» -> короткий числовой отпечаток, который Python может посчитать и использовать как адрес в каталоге. Этот отпечаток называется хеш (hash), а объект, для которого его можно посчитать корректно и надёжно, называют хешируемым (hashable).

Главное практическое правило звучит так: хешируемый объект обязан иметь хеш, который не меняется, пока объект лежит внутри множества или используется как ключ словаря. Почему это важно? Потому что множество и словарь сначала идут «по адресу» (по хешу), а уже потом уточняют равенство. Если бы хеш мог поменяться по ходу дела, элемент оказался бы как будто «переехал на другую полку», и структура данных потеряла бы способность находить его быстро и правильно.

Отсюда и понятная связь с неизменяемостью. Большинство изменяемых типов (например, `list`, `dict`, `set`) нельзя хешировать, потому что их содержимое меняется, а вместе с ним менялась бы и логика поиска. Поэтому такие объекты нельзя класть в множество и нельзя использовать как ключ словаря. А неизменяемые типы (например, `int`, `str`, `tuple` при условии, что внутри тоже только хешируемые элементы) обычно хешируемы, и поэтому прекрасно "живут" в `set`.

Есть удобная проверка: если объект можно использовать ключом словаря, то он хешируемый, а значит подходит и для множества.

```
ok = {}
ok["hello"] = 1
ok[(1, 2)] = 2
```

И наоборот, пример типичной ошибки: кортеж сам по себе выглядит «неизменяемым», но если внутри спрятан список, то весь кортеж становится не хешируемым.

```
myset = {1, 2, (5, 6), 7}       # так можно
myset = {1, 2, ([5, 6]), 7}     # TypeError: внутри кортежа список
```

Если запомнить одну мысль, пусть будет эта: множество принимает только такие элементы, которые умеют иметь стабильный «адрес» для быстрого поиска. Это и есть смысл **hashable**.

Что такое множество в Python

Множество (set) в Python это неупорядоченная коллекция без повторов. Если вы добавляете одинаковые элементы, «лишнее» просто исчезает.

Очень важно говорить точным языком. Правило не про «неизменяемость как философию», а про хешируемость (hashable) как техническое требование. Элемент множества должен быть таким, чтобы Python мог вычислить для него хеш и надёжно находить его внутри структуры. На практике у встроенных типов это обычно означает: числа, строки и кортежи (если внутри кортежа тоже только подходящие элементы) отлично подходят, а списки и сами множества не подходят.

Создание множества

Начнём с самого наглядного способа: фигурные скобки с элементами через запятую.

```
numbers = {10, 20, 30, 40, 50}
fruits = {"apple", "banana", "cherry"}


А вот момент, на котором спотыкаются почти все новички: пустое множество
создаётся не фигурными скобками, а через set(). Потому что {} в Python
зарезервировано под пустой словарь.

empty_set = set()
empty_set = {}  # Это словарь, а не множество
```

Запомните это как «правило первой помощи»: увидели `{}` без элементов -> это словарь. Нужен пустой set -> пишем `set()`.

Вывод множества

Множество не обещает порядок при обходе и печати. Поэтому один и тот же набор элементов может выглядеть по-разному на экране, и это не ошибка, а нормальное поведение.

```
numbers = {10, 20, 30, 40, 50}
fruits = {"apple", "banana", "cherry"}

print(numbers)  # Выведет: {10, 20, 30, 40, 50}
print(fruits)   # Выведет: {'banana', 'apple', 'cherry'}
```

Хитрость в том, что комментарий в таких примерах показывает один из возможных вариантов. Вывод может прийти в другом порядке, и ваш код от этого не становится неправильным.

Встроенная функция set()

Функция `set()` умеет собирать множество из любой итерируемой последовательности. Это удобно, когда исходные данные уже где-то есть: в списке, строке или кортеже. Вы передаёте «что угодно, по чему можно пройтись», а на выходе получаете уникальные элементы.

```
my_set = set([1, 2, 3, 4, 5])  # Множество из списка
```

```
my_set2 = set("hello")         # Множество из строки
```

Со строками особенно наглядно: повторы исчезают, остаются только уникальные символы.

И да, пустое множество можно получить и так. Это корректно, просто обычно пишут короче через `set()`.

```
empty_set = set([])  # Пустое множество из пустого списка
```

Дубликаты при создании множества

Множество устроено так, что повторы не имеют права на жизнь. Вы можете передать хоть десять одинаковых единиц, и в результате всё равно останется одна.

```
my_set = set([1, 1, 2, 3, 3])
print(my_set)  # Выведет: {1, 2, 3}


my_set = set("aabbcc")
print(my_set)  # Выведет: {'a', 'b', 'c'}
```

Если вы увидели перестановку элементов, не пытайтесь «чинить порядок». Это не баг, это природа множества.

Типы данных в множествах

Теперь главный технический закон: элементы множества должны быть хешируемыми. В учебной практике это удобно запоминать так: «как правило, это неизменяемые вещи». Числа, строки, кортежи подходят. Списки и множества внутрь класть нельзя.

```
valid_set = {1, 2, (3, 4), "hello"}  # Все элементы могут быть в множестве
invalid_set = {1, 2, [3, 4]}         # Ошибка, список не может быть элементом
множества
```

Обратите внимание на вторую строку: это пример «как делать нельзя». Если попытаться выполнить такой код, Python честно остановит программу с `TypeError`.

Ограничения на кортежи в множествах

Кортежи в множествах разрешены, но только при одном условии: внутри кортежа тоже не должно быть «запрещённых» элементов. Логика простая: множество требует хешируемость, кортеж будет хешируемым только тогда, когда все его части тоже хешируемы.

```
myset = {1, 2, (5, 6), 7}  # Множество может содержать кортеж
```

Этот пример допустим, потому что `(5, 6)` состоит из целых чисел.

А вот здесь ловушка для новичков: кортеж с вложенным списком становится проблемой, потому что список внутри не хешируемый.

```
myset = {1, 2, ([5, 6]), 7}  # Ошибка: TypeError, список внутри кортежа
```

Частые ошибки и нюансы

Если держать в голове один принцип, пусть это будет принцип хешируемости. Как только вы

пытаетесь положить внутрь множества список или другое множество, вы почти гарантированно получаете `TypeError`.

```
myset1 = {1, 2, [5, 6], 7}  # Ошибка: TypeError, список не может быть элементом множества
myset2 = {1, 2, {5, 6}, 7}  # Ошибка: TypeError, множество не может быть элементом множества
```

И наоборот, кортеж часто становится аккуратным способом «упаковать пару значений» так, чтобы множество могло это хранить.

```
myset = {1, 2, (5, 6), 7}  # Множество может содержать кортеж
```

Работа с f-строками

Иногда хочется вывести множество «красиво» и в предсказуемом порядке. Само множество порядок не гарантирует, поэтому идея такая: сначала получить отсортированный список элементов, затем склеить их в строку, а фигурные скобки вывести как обычный текст.

```
print(f"Уникальные цифры: {{{', '.join(sorted(unique_digits))}}}")
```

Здесь всё держится на трёх шагах. `sorted(unique_digits)` даёт элементы в стабильном порядке. `', '.join(...)` превращает их в одну строку через запятую. А `{{` и `}}` в f-строке печатают символы `{` и `}` буквально, потому что одинарные скобки зарезервированы под подстановки.

В результате получается «аккуратная» строка с предсказуемым порядком, например:

Уникальные цифры: {0, 1, 3, 5, 7}

Заключение

Множества берите там, где нужна уникальность и быстрые проверки вхождения. Пустое множество создавайте через `set()`, чтобы не перепутать со словарём. Следите за типами элементов: внутри должны быть хешируемые значения, иначе получите `TypeError`. И не рассчитывайте на порядок вывода: если важна последовательность, выбирайте список.

9.2 Множества: Основные операции и функции

Работа с множествами

Основы работы с множествами

Давай начнём с идеи: множество в Python хранит элементы без повторов. Похоже на список по ощущениям, но есть важное отличие: каждый объект в множестве уникален. Поэтому если тебе важно убрать дубли и быстро проверять наличие элемента -> множества очень кстати

Функция len()

Частая задача: понять, сколько разных значений у нас есть. Для этого спокойно используем `len()`

Пример:

```
myset1 = {3, 3, 5, 7, 7}
myset2 = set([1, 3, 3, 4, 5, 5])
myset3 = set('ppqqrrsstt')

print(len(myset1))  # Выведет: 3
print(len(myset2))  # Выведет: 4
print(len(myset3))  # Выведет: 5
```

Пояснение: `len()` считает именно разные элементы, дубликаты автоматически отбрасываются

Оператор принадлежности in

Хочешь быстро узнать: «есть ли это значение в наборе» -> используй `in`. Для множеств такая проверка работает очень быстро

Пример:

```
numbers = {1, 3, 5, 7, 9}

if 3 in numbers:
    print('Множество numbers содержит число 3')
else:
    print('Множество numbers не содержит число 3')
```

Вывод:

```
Множество numbers содержит число 3
```

Ещё удобно проверять отсутствие через `not in`

Пример:

```
numbers = {1, 3, 5, 7, 9}

if 0 not in numbers:
    print('Множество numbers не содержит числа 0')
```

Пояснение: проверка `in` для множеств обычно быстрее, чем для списков, поэтому множества хорошо подходят для частых «поисков по содержимому»

Встроенные функции sum(), min(), max()

Если в множестве лежат числа, можно сразу считать сумму, минимум и максимум

- `sum()` складывает все элементы
- `min()` даёт наименьший элемент
- `max()` даёт наибольший элемент

Пример:

```
numbers = {1, 3, 5, 7}

print('Сумма элементов множества =', sum(numbers))  # Выведет: Сумма элементов
множества = 16
print('Минимальный элемент =', min(numbers))      # Выведет: Минимальный элемент
= 1
print('Максимальный элемент =', max(numbers))     # Выведет: Максимальный
элемент = 7
```

Пояснение: базовые вычисления по множеству делаются в одну строку -> удобно для быстрой аналитики числовых данных

Перебор множества

Перебирать элементы можно обычным `for`. Важно: множества не хранят порядок, поэтому порядок обхода может отличаться от запуска к запуску

Пример:

```
fruits = {"apple", "banana", "cherry"}

for fruit in fruits:
    print(fruit)
```

Пояснение: не рассчитывай на фиксированный порядок вывода, у множеств его нет по определению

Форматированный вывод множеств

Нужно красиво показать элементы на экране: соберём их в строку

Пример:

```
myset = {1, 2, 3, 4}

# Форматированный вывод с использованием f-строки
print(f"Множество содержит элементы: {', '.join(map(str, myset))}")
```

Пояснение: `', '.join(map(str, myset))` превращает каждый элемент в строку и

соединяет их через запятую

Сравнение множеств

Сравнивать множества можно по «содержимому»: равны они или нет

Пример:

```
set1 = {1, 2, 3}
set2 = {3, 2, 1}
set3 = {4, 5, 6}

print(set1 == set2)  # Выведет: True, так как множества содержат одни и те же элементы
print(set1 != set3)  # Выведет: True, множества разные
```

Пояснение: порядок не важен, важен состав элементов

Функция sorted()

Хочется увидеть элементы в порядке возрастания: отсортируем. Помни: `sorted()` вернёт список, не множество

Пример:

```
myset = {4, 1, 3, 2}

sorted_set = sorted(myset)
print(sorted_set)  # Выведет: [1, 2, 3, 4]
```

Пояснение: множества сами по себе неупорядочены, поэтому сортировка всегда возвращает новый список

Сортировка с помощью key

Иногда нужно правило посложнее. Например: сортировать строки по длине, а при равной длине -> по алфавиту. Для этого используем параметр `key`

Пример:

```
resources = {"banana", "apple", "pear", "kiwi", "fig"}

sorted_resources = sorted(resources, key=lambda x: (len(x), x))
print(sorted_resources)
# Выведет: ['fig', 'kiwi', 'pear', 'apple', 'banana']
```

Как это работает:

- функция `lambda x: (len(x), x)` строит кортеж признаков, до `lambda` мы скоро доберемся
- сначала сравнивается длина строки
- если длины равны -> сравниваются сами строки по алфавиту

Обратная сортировка (reverse=True)

Нужно в убывающем порядке: добавляем `reverse=True`

Пример:

```
numbers = {10, 3, 7, 1}

desc_sorted = sorted(numbers, reverse=True)
print(desc_sorted)  # Выведет: [10, 7, 3, 1]
```

Можно комбинировать `key` и `reverse`. Например: строки по убыванию длины

Пример:

```
words = {"cat", "elephant", "dog", "bee"}

longest_first = sorted(words, key=len, reverse=True)
print(longest_first)
# Выведет: ['elephant', 'cat', 'dog', 'bee']
```

Заключение

Мы разобрали самые полезные приёмы: как считать размер множества, быстро проверять наличие элемента, делать базовые вычисления, перебирать и красиво выводить значения, сравнивать множества и сортировать элементы. Возьми это как набор инструментов: для уникальных значений множества дают простоту и скорость, а всё остальное делается понятными функциями Python

9.3 Методы добавления и удаления элементов в множества

Методы множеств

Добавление элементов

Давай продолжим: множество у нас уже есть, теперь научимся добавлять новые значения. Это простой и частый приём: «был набор» -> «добавили ещё что-то»

1 Метод add(): добавляет один элемент в множество

Пример:

```
numbers = {1, 2, 3, 4}
numbers.add(10)  # Добавляем число 10 в множество
numbers.add(15)  # Добавляем число 15 в множество

print(numbers)
```

Вывод:

```
{1, 2, 3, 4, 10, 15}
```

Пояснение: множества неупорядочены, поэтому порядок элементов в выводе может быть произвольным

Примечание: `add()` ничего не меняет, если такой элемент уже есть в множестве

2 Добавление нескольких элементов: удобно пройтись циклом

Пример:

```
numbers = set()  # Пустое множество

for i in range(5, 15, 2):
    numbers.add(i)

print(numbers)
```

Вывод:

```
{5, 7, 9, 11, 13}
```

Пояснение: здесь мы добавляем только нечётные числа из диапазона 5..14

Удаление элементов

Удалять можно по-разному: `remove()`, `discard()`, `pop()`. У каждого метода свой характер: один строгий, другой мягче, третий возвращает удалённый элемент

1 Метод remove(): удаляет указанный элемент, если его нет -> ошибка

Пример:

```
numbers = {1, 2, 3, 4, 5}
numbers.remove(3)
print(numbers)  # Выведет: {1, 2, 4, 5}

# Попытка удалить элемент, которого нет в множестве:
```

```
numbers.remove(10)  # Ошибка: KeyError
```

Пояснение: если элемента нет, `remove()` поднимает `KeyError`

2 Метод discard(): ведёт себя мягче, ошибки не будет, даже если элемента нет

Пример:

```
numbers = {1, 2, 3, 4, 5}
numbers.discard(4)
print(numbers)  # Выведет: {1, 2, 3, 5}

numbers.discard(10)  # Не вызывает ошибку, даже если элемента нет
print(numbers)  # Выведет: {1, 2, 3, 5}
```

Пояснение: отличие от `remove()` как раз в отсутствии исключения

3 Метод pop(): удаляет и возвращает случайный элемент, на пустом множестве -> ошибка

Пример:

```
symbols = {"a", "b", "c", "d", "e"} # numbers = {1, 2, 3, 4, 5} почему не такое
множество на примере?

removed_element = symbols.pop()
print(f"Удалённый элемент: {removed_element}")
print(f"Оставшееся множество: {symbols}")
```

Примечание: порядок элементов в множестве может выглядеть «сохранённым» при выводе, но множества по определению неупорядочены и ориентироваться на порядок не стоит

Метод очистки множества

Нужно быстро освободить множество: `clear()` убирает всё содержимое

Пример:

```
numbers = {1, 2, 3, 4, 5}
numbers.clear()
print(numbers)  # Выведет: set()
```

Пояснение: после `clear()` множество пустое. Пустое множество печатается как `set()`, потому что `{}` зарезервировано под пустой словарь

Заключение

Мы разобрали базовые операции управления содержимым множества: как добавлять (`add()` и через цикл), как удалять строго (`remove()`), мягко (`discard()`), а также как забрать и вернуть удалённый элемент (`pop()`) и как очистить всё (`clear()`). Эти приёмы помогают гибко обновлять набор уникальных значений и держать коллекцию аккуратной и полезной для задач анализа данных

9.4 Методы объединения, пересечения и разности множеств

Операции и методы множеств

Операции над множествами

Давай соберём в одном месте базовые операции: объединение, пересечение, разность и симметрическая разность. У каждой есть метод и есть оператор: результат один и тот же, меняется только запись. Посмотрим обе формы и сравним, где удобнее использовать каждую

Объединение множеств: метод union()

Объединение берёт все элементы из обоих множеств: повторов не будет, останутся только уникальные значения

Пример:

```
set_1 = {1, 2, 3, 4}
set_2 = {3, 4, 5, 6}

set_3 = set_1.union(set_2)
print(set_3)
```

Вывод:

```
{1, 2, 3, 4, 5, 6}
```

Пояснение: `union()` создаёт новое множество, исходные не меняет

Ту же операцию можно записать короче с оператором `|`

Пример:

```
set_3 = set_1 | set_2
print(set_3)
```

Вывод будет аналогичен

Пересечение множеств: метод intersection()

Пересечение оставляет только общие элементы, которые есть и в первом, и во втором множестве

Пример:

```
set_1 = {1, 2, 3, 4}
set_2 = {3, 4, 5, 6}

set_3 = set_1.intersection(set_2)
print(set_3)
```

Вывод:

```
{3, 4}
```

Ту же операцию можно записать оператором `&`

Пример:

```
set_3 = set_1 & set_2
print(set_3)
```

Результат будет тот же

Разность множеств: метод difference()

Разность берёт элементы первого множества, которых нет во втором

Пример:

```
set_1 = {1, 2, 3, 4}
set_2 = {3, 4, 5, 6}

set_3 = set_1.difference(set_2)
print(set_3)
```

Вывод:

```
{1, 2}
```

То же самое с оператором −

Пример:

```
set_3 = set_1 - set_2
print(set_3)
```

Вывод будет аналогичен

Обрати внимание: порядок операндов важен. Поменяешь множества местами -> изменится и результат

Симметрическая разность: метод symmetric_difference()

Симметрическая разность возвращает элементы, которые встречаются только в одном из множеств, но не сразу в обоих

Пример:

```
set_1 = {1, 2, 3, 4}
set_2 = {3, 4, 5, 6}

set_3 = set_1.symmetric_difference(set_2)
print(set_3)
```

Вывод:

```
{1, 2, 5, 6}
```

Ту же операцию можно записать оператором ^

Пример:

```
set_3 = set_1 ^ set_2
print(set_3)
```

Результат будет аналогичен

Методы для изменения множества

Методы `union()`, `intersection()`, `difference()`, `symmetric_difference()` возвращают новое множество и не трогают исходные. Если нужно изменить текущее множество на месте, используй «*_update»-методы

`update()` -> добавляет элементы другого итерируемого объекта в текущее множество
`intersection_update()` -> оставляет только общие элементы
`difference_update()` -> удаляет из текущего множества все элементы, которые есть в другом
`symmetric_difference_update()` -> оставляет только элементы, которые есть ровно в одном из множеств

Ниже разберём каждый метод подробно

Метод update()

`update()` добавляет элементы из другого множества или любого итерируемого объекта в текущее множество

Синтаксис:

```
set1.update(iterable)
```

`iterable` -> любой итерируемый объект: множество, список, кортеж, строка

Пример:

```
myset = {1, 2, 3}
myset.update([3, 4, 5, 6])  # добавляем элементы из списка, существующие игнорируются

print(myset)
```

Вывод:

```
{1, 2, 3, 4, 5, 6}
```

Пояснение: после `update()` в `myset` собраны все уникальные элементы из множества и списка

Можно передать и другие итерируемые, например строку

```
myset.update("abc")
print(myset)  # Выведет: {1, 2, 3, 4, 5, 6, 'a', 'b', 'c'}, порядок букв может отличаться
```

Метод intersection_update()

`intersection_update()` оставляет в текущем множестве только те элементы, которые есть и в нём, и в другом итерируемом объекте

Синтаксис:

```
set1.intersection_update(iterable)
```

`iterable` -> итерируемый объект для пересечения

Пример:

```
set_1 = {1, 2, 3, 4, 5}
```

```
set_2 = {3, 4, 5, 6, 7}

set_1.intersection_update(set_2)

print(set_1)
```

Вывод:

```
{3, 4, 5}
```

Пояснение: остаются только общие элементы

Эквивалентная запись через оператор:

```
set_1 &= set_2
```

Этот код даст тот же результат

Метод difference_update()

`difference_update()` удаляет из текущего множества все элементы, которые встречаются во втором итерируемом объекте

Синтаксис:

```
set1.difference_update(iterable)
```

`iterable` -> элементы из него будут удалены из текущего множества

Пример:

```
set_1 = {1, 2, 3, 4, 5}
set_2 = {3, 4, 5, 6, 7}

set_1.difference_update(set_2)

print(set_1)
```

Вывод:

```
{1, 2}
```

Пояснение: из `set_1` исчезло всё, что было в `set_2`

Эквивалент через оператор:

```
set_1 -= set_2
```

Этот код также приведёт к результату `{1, 2}`

Метод symmetric_difference_update()

`symmetric_difference_update()` оставляет только те элементы, которые есть ровно в одном из множеств

Синтаксис:

```
set1.symmetric_difference_update(iterable)
```

`iterable` -> объект для симметрической разности

Пример:

```
set_1 = {1, 2, 3, 4, 5}
set_2 = {3, 4, 5, 6, 7}

set_1.symmetric_difference_update(set_2)

print(set_1)
```

Вывод:

```
{1, 2, 6, 7}
```

Пояснение: остались элементы, которые не пересекались

Эквивалент через оператор:

```
set_1 ^= set_2
```

Этот код также даст тот же результат

Множества в Python, с их определениями и операторами:

Оператор/Метод -> Описание

A | B / A.union(B) -> объединение множеств A и B
A |= B / A.update(B) -> добавить в A все элементы из B
A & B / A.intersection(B) -> пересечение множеств A и B
A &= B / A.intersection_update(B) -> оставить в A только элементы, которые есть в B
A - B / A.difference(B) -> разность множеств A и B
A -= B / A.difference_update(B) -> удалить из A все элементы, входящие в B
A ^ B / A.symmetric_difference(B) -> симметрическая разность множеств A и B
A ^= B / A.symmetric_difference_update(B) -> записать в A симметрическую разность множеств A и B

Оператор -> Описание

- -> разность
& -> пересечение
^ -> симметрическая разность
| -> объединение

В виде таблицы:

Оператор/Метод	Описание
`A	B`	`A.union(B)` Возвращает множество, являющееся объединением множеств `A` и `B`
`A |= B`	`A.update(B)` Добавляет в множество `A` все элементы из множества `B`

Оператор/Метод	Описание
`A & B`	`A.intersection(B)` Возвращает множество, являющееся пересечением множеств A и B.
`A &= B`	`A.intersection_update(B)` Оставляет в множестве A только те элементы, которые есть в множестве B.
`A - B`	`A.difference(B)` Возвращает разность множеств A и B.
`A -= B`	`A.difference_update(B)` Удаляет из множества A все элементы, входящие в B.
`A ^ B`	`A.symmetric_difference(B)` Возвращает симметрическую разность множеств A и B.
`A ^= B`	`A.symmetric_difference_update(B)` Записывает в A симметрическую разность множеств A и B.

Оператор	Описание
`-`	разность
`&`	пересечение
`^`	симметрическая разность
`|`	объединение

Эта таблица включает наиболее распространенные операции и методы для работы с множествами.

Заключение

Теперь у тебя под рукой полный набор операций для работы с множествами: методы и операторы для объединения, пересечения, разности и симметрической разности, плюс версии «на месте» через *_update. Пользуйся операторной записью, когда хочется короче, и методами, когда важнее читаемость и явность действия. Освоив эти приёмы, ты свободно собираешь, фильтруешь и обновляешь наборы уникальных значений без лишнего кода

9.5 Методы множеств: Подмножества, Надмножества и Сравнение

Методы множеств и их сравнение

Что такое подмножества и надмножества

Давай зафиксируем термины: подмножество -> все его элементы уже есть в другом множестве. Если A подмножество B, то каждый элемент A содержится в B.
Надмножество -> наоборот: A включает в себя все элементы B.
Нестрогое подмножество допускает равенство: {1, 2, 3} считается нестрогим подмножеством самого себя

Метод issubset(): проверка подмножества

`issubset()` отвечает на вопрос: «все ли мои элементы есть во втором множестве» -> True/False

Пример:

```
group_a = {4, 8, 15}
group_b = {4, 8, 15, 16, 23, 42}

print(group_a.issubset(group_b))  # Вывод: True
```

Здесь `group_a` действительно «внутри» `group_b`

Можно короче через операторы:

- <= -> нестрогое подмножество
- < -> строгое подмножество

Пример:

```
group_a = {4, 8, 15}
group_b = {4, 8, 15, 16, 23, 42}

print(group_a <= group_b)  # Вывод: True
print(group_a < group_b)   # Вывод: True
```

Важно: < требует, чтобы множества отличались по составу

Метод issuperset(): проверка надмножества

`issuperset()` проверяет обратную ситуацию: «содержу ли я все элементы другого множества»

Пример:

```
team_a = {'John', 'Sarah', 'David'}
team_b = {'David', 'Sarah'}

print(team_a.issuperset(team_b))  # Вывод: True
```

Операторы работают аналогично, только «в другую сторону»:

- >= -> нестрогое надмножество
- > -> строгое надмножество

Пример:

```
team_a = {'John', 'Sarah', 'David'}
team_b = {'David', 'Sarah'}

print(team_a >= team_b)  # Вывод: True
print(team_a > team_b)   # Вывод: True
```

Совет: когда важна краткость записи -> используй операторы, когда хочется явности -> методы

Метод isdisjoint(): проверка на отсутствие общих элементов

`isdisjoint()` отвечает: «пересекаются ли множества». Не пересекаются -> True, есть общий элемент -> False

Пример:

```
students_a = {101, 102, 103, 104}
students_b = {105, 106, 107}

print(students_a.isdisjoint(students_b))  # Вывод: True
```

Пример с пересечением:

```
students_a = {101, 102, 103, 104}
students_b = {103, 105, 106}

print(students_a.isdisjoint(students_b))  # Вывод: False
```

Обрати внимание: достаточно одного общего элемента, чтобы получить `False`

Сравнение множества с другими итерируемыми объектами

Методы `issubset()`, `issuperset()`, `isdisjoint()` вызываются у множества, но вторым аргументом может быть любой итерируемый объект: список, кортеж, строка

Пример:

```
set_a = {1, 2, 3}
list_b = [1, 2, 3, 4, 5]

print(set_a.issubset(list_b))  # Вывод: True
```

Пояснение: метод проверяет, что все элементы `set_a` встречаются в `list_b`, преобразовывая его к множеству «под капотом»

Соответствия методов и операторов

Операция -> Метод -> Описание
`set_a <= set_b` -> `set_a.issubset(set_b)` -> True, если `set_a` подмножество

set_b
set_a >= set_b -> set_a.issuperset(set_b) -> True, если set_a надмножество set_b
set_a < set_b -> — -> строгое подмножество: set_a <= set_b и set_a != set_b
set_a > set_b -> — -> строгое надмножество: set_a >= set_b и set_a != set_b

Пояснения:
<= и >= опираются на issubset() и issuperset()
строгие варианты < и > пишутся операторами, прямых методов под них нет

? В виде таблицы соответствия методов и операторов

Операция	Метод	Описание
set_a <= set_b	set_a.issubset(set_b)	Возвращает True, если set_a является подмножеством set_b.
set_a >= set_b	set_a.issuperset(set_b)	Возвращает True, если set_a является надмножеством set_b.
set_a < set_b	—	Строгое подмножество: set_a <= set_b и set_a != set_b.
set_a > set_b	—	Строгое надмножество: set_a >= set_b и set_a != set_b.

Пояснения:

- Операторы <= и >= используют методы issubset() и issuperset(), соответственно, для проверки, является ли одно множество подмножеством или надмножеством другого.
- Строгое подмножество (<) и строгое надмножество (>) не имеют прямого метода в Python, но они могут быть проверены с помощью комбинированных условий (например, set_a <= set_b и set_a != set_b).

Заключение

Теперь у тебя есть быстрый набор проверок: подмножество, надмножество и «не пересекаются ли». Методы дают читаемость, операторы позволяют писать короче. Пользуйся тем вариантом, который делает код яснее в твоей задаче: фильтрация групп, проверка включения и поиск пересечений получаются буквально в одну строку